

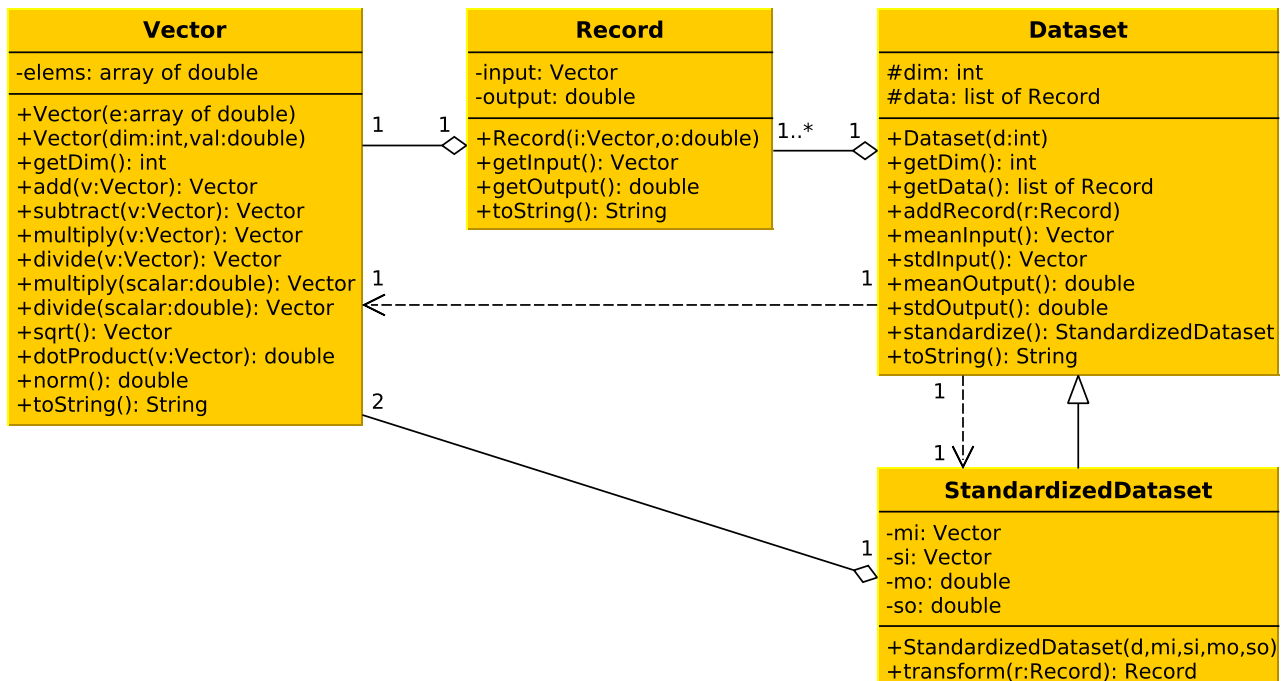
Lab 2: Implementing a class diagram

24292-Object Oriented Programming

1 Introduction

The aim of this lab is to implement the design of Seminar 2 consisting of a larger class diagram. The session is mandatory and you have to deliver the source code of the Java project as well as a document that outlines the solution of the problem. To elaborate the document you can use the guidelines from Lab 1.

You are free to use your own solution to Seminar 2, but for reference, a sample solution appears below.



2 The TestDataset class

As before, define a `TestDataset` class with a main method where the other classes are tested. Frequently test the classes in the main method by creating instances and calling methods on them. As before, your main method should always test all the methods of the class definitions to make sure that they are correct.

3 The classes Vector and Record

The first step is to incorporate the classes `Vector` and `Record` from Lab 1. To do so you can simply copy them to a new directory. In principle it is not necessary to modify these classes, unless you discover an error in the code (sometimes new errors are discovered after running more complicated tests). Since you already tested these classes in Lab 1, it should not be necessary to test the individual classes again.

4 The Dataset class

Next implement the `Dataset` class. The class has an aggregation relation with `Record` due to the attribute `data` which permanently stores references to instances of `Record`. In contrast, no instance of `Vector` is stored as part of an attribute, but there are two methods that create and return vectors, so the relation with `Vector` is of type use/dependency.

The existing Java class `ArrayList` is a good choice of collection for storing the records, since this class allows accessing a record at an arbitrary index in constant time. The intention is to create a `Dataset` with an empty list of records, and add records one by one using the `addRecord` method. The two getters `getDim` and `getData` should be straightforward to implement.

To implement methods for computing the mean μ and standard deviation σ of n values $x_1, \dots, x_n$, you need the following definitions:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i = \frac{1}{n} (x_1 + \dots + x_n),$$
$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2} = \sqrt{\frac{1}{n} ((x_1 - \mu)^2 + \dots + (x_n - \mu)^2)}.$$

Note that the mean and standard deviation applies individually to each input dimension. Instead of writing code for the individual input dimensions, both the mean and standard deviation of the input can be compactly computed by combining the existing methods of the `Vector` class.

You do not have to implement the method `standardize` yet, since this method requires first writing the class definition of `StandardizedDataset`. Create an instance of `Dataset` in the main method and call methods on this instance to convince yourself that the class works as intended.

4.1 The StandardizedDataset class

The StandardizedDataset class is related to Dataset using two relations. To exploit the fact that an instance of StandardizedDataset is also a dataset, StandardizedDataset inherits from Dataset using the inheritance relation. Moreover, the Dataset class creates and returns an instance of StandardizedDataset in the `standardize` method, which corresponds to a relation of use/dependency. Unlike Dataset, StandardizedDataset has an aggregation relation with Vector since the class stores several references to instances of Vector in the attributes.

Due to inheritance, all attributes and methods of Dataset are also part of StandardizedDataset. Hence it is not necessary to implement these attributes and methods again. Compared to Dataset, StandardizedDataset has four additional attributes corresponding to the mean and standard deviation of the inputs, and the mean and standard deviation of the outputs. In addition, the class needs a method `transform` that transforms an instance of Record according to the description from Seminar 2 and returns the transformed record.

Once you have finished implementing StandardizedDataset, you can now implement the method `standardize` of Dataset. This method should compute the mean and standard deviation of the inputs and outputs, create an instance of StandardizedDataset, transform each record using the `transform` method, add the new records to the instance of StandardizedDataset, and finally return the instance of StandardizedDataset. A good way to test that the code is correct is to create a dataset, apply the method `standardize` to create a standardized dataset, and compute the mean and standard deviation for the standardized dataset (the mean should be 0 and the standard deviation should be 1).

5 Optional: normalized datasets

An alternative to standardization is *normalization*, in which the inputs and outputs are transformed to have values in the range $[0, 1]$. To perform normalization of n values $x_1, \dots, x_n$ we have to find the maximum and minimum value and rescale the values such that the minimum corresponds to 0 and the maximum to 1. Implement a class NormalizedDataset as well as the code necessary for performing normalization of a dataset, analogous to standardization. (Please do not implement linear regression yet since this is the topic of the next seminar and lab.)

6 Submission

The source code and documentation should all be uploaded to a directory **Lab2** of your Git repository **prior to the next lab**. Please make sure that the directory name is *exactly* Lab2 (not Lab_2, Lab-2, LAB2, lab2 etc.), and that Lab2 is located in the **root of the Git repository**. Failure to comply with these rules will result in a grade deduction.